

Índice

Guía de estudio para el primer parcial — versión <i>full</i> (con respuestas)	1
Parte 1 — Repaso y profundización de temas previos	1
1. ¿Qué es un objeto? / Repaso del paradigma	1
2. Repaso de subclasificación	3
Parte 2 — Esquemas no tradicionales para definir el comportamiento	4
3. Mixines	4
4. Traits	6
5. Closures	8
Parte 3 — Metaprogramación	9
6. Introducción a la metaprogramación	10
7. Aspectos estructurales de los metamodelos	12
8. Formas de representar y modificar el comportamiento dinámicamente	14
9. Comparación de metamodelos y maneras de entender la programación	16
Apéndice — Comentarios sobre la subclasificación (resuelto)	17
Lista de lecturas (referencia rápida)	18

Guía de estudio para el primer parcial — versión *full* (con respuestas)

Programación con Objetos 3 — Universidad Nacional de Quilmes

Este documento toma cada ítem de la *Guía de estudio para el primer parcial (2026s1)* y le agrega una respuesta desarrollada, usando exclusivamente el material de la cátedra (diapositivas de clase, apuntes y bibliografía de la carpeta). Al final de cada respuesta se indica **la referencia** de donde se sacó el contenido. Cuando un punto no tenía bibliografía específica en la carpeta, se usó la documentación oficial de Ruby y se aclara explícitamente.

Recordá lo que dice la guía: el parcial **no** evalúa reproducción mecánica ni sintaxis de Ruby, sino que manejes los conceptos y uses el vocabulario con precisión.

Parte 1 — Repaso y profundización de temas previos

1. ¿Qué es un objeto? / Repaso del paradigma

1.1. Análisis crítico de distintas definiciones de “objeto”

Existen varias definiciones de “objeto” que conviene comparar críticamente:

- **Definición técnica/computacional:** “los objetos son los elementos computacionales que nos ofrece el paradigma de objetos para construir programas”. Son la “célula” con la que componemos software. Es útil pero no conecta el programa con la realidad que representa.
- **Definición conceptual:** “Un objeto es una representación esencial de una cosa de un dominio de interés”. Es más rica porque liga el programa con el dominio, pero exige a su vez definir tres términos: *cosa* (todo aquello acerca de lo cual podemos decir algo), *dominio de interés* (el conjunto de cosas relevantes para el problema) y *representación esencial* (un modelo no arbitrario que captura la esencia, es decir, lo que hace que la cosa sea lo que es y no otra).

- **Definiciones “de la web”:** suelen apoyarse en palabras no definidas (“una cosa”, “una entidad”, “un algo”) o reducir el objeto a “datos + procedimientos”, lo cual contamina el paradigma con conceptos ajenos (datos, funciones) que el paradigma justamente busca no presuponer.

La crítica central de la cátedra es que la mejor definición es **pura** (no menciona conceptos externos al paradigma como “datos”, “if”, “funciones”) y **minimal** (no introduce elementos superfluos o deducibles, como “clase” o “herencia”). Por eso se prefiere derivar todo a partir de la noción de mensaje.

Referencia: *Algunas definiciones.pdf* (slides Clase 1: “Objeto (de Objetos 1)”, “Objeto”, “Programa”); 2. *Conceptos fundamentales de la programación con objetos.pdf* (Material adicional, secciones “¿Qué es un objeto?”, “¿Qué entendemos por cosa?”, “representación esencial”).

1.2. El concepto de “objeto” definido a partir de la noción de “mensaje”

El punto de partida es la **definición de programa**: “Un programa orientado a objetos es un conjunto de objetos que colaboran enviándose mensajes”. De ahí salen dos postulados:

1. **Sólo hay objetos:** no existen elementos computacionales de otra naturaleza (no hay “datos”, “funciones”, “estructuras de control”).
2. **Lo único que pueden hacer los objetos es enviar y recibir mensajes:** toda interacción ocurre a través de mensajes.

De estos postulados se deriva la caracterización del objeto centrada en mensajes: *la característica más importante de los objetos es que podemos pedirles que hagan cosas por nosotros, enviándoles mensajes; cada objeto reacciona y decide cómo reaccionar*. Un objeto queda entonces definido por **su protocolo** (el conjunto de mensajes que entiende), que se corresponde con lo que se puede hacer con la cosa que representa. Un mensaje es “la manera en que un objeto le solicita a otro que haga algo”: especifica el **qué**, no el **cómo**.

Referencia: 2. *Conceptos fundamentales...pdf* (“¿Qué es un programa?”, postulados 1 y 2; “¿Qué es un mensaje?”); *Algunas definiciones.pdf* (slide “Objeto”).

1.3. Propiedades deducibles: identidad, implementación interna y polimorfismo

A partir de la definición basada en mensajes se deducen tres propiedades:

- **Identidad:** como las colaboraciones son *dirigidas* (tengo que poder decir “a *este* objeto le mando el mensaje”), cada objeto necesita una identidad única que lo distinga de todos los demás, incluso de otros con el mismo comportamiento e implementación. Hay una razón **técnica** (poder dirigir mensajes y establecer relaciones de conocimiento) y una **conceptual** (cada cosa del dominio es idéntica a sí misma y sólo a sí misma — *principio de identidad* —, así que su representación también debe tenerlo). La identidad **no** debe confundirse con la **igualdad**, que es relativa y depende de criterios arbitrarios que nosotros definimos (en Smalltalk, `==` compara identidad y `=` compara igualdad, redefinible por cada objeto).
- **Implementación interna (privada) / encapsulamiento:** el objeto tiene un **comportamiento** (el qué, su protocolo público) y una **implementación** para ese comportamiento (el cómo: sus *métodos* y sus *colaboradores internos*). La implementación es **privada**: ningún objeto externo puede ni debe conocerla o depender de ella, y nadie puede “forzar” la ejecución de un método ajeno (sólo el propio objeto ejecuta sus métodos al recibir el mensaje). Esta separación entre el qué y el cómo es el **encapsulamiento**, y permite cambiar la implementación sin afectar a los demás mientras se respete el protocolo.
- **Polimorfismo:** como lo único que se puede hacer con un objeto es enviarle mensajes, y la implementación está oculta, dos objetos que entienden el mismo mensaje resultan **intercambiables** desde el punto de vista del emisor. El emisor depende del protocolo, no de la implementación, así que objetos distintos pueden responder al mismo mensaje cada uno a su manera. El polimorfismo es, entonces, una consecuencia directa de definir al objeto por su protocolo y de la privacidad de la implementación.

Referencia: 2. *Conceptos fundamentales...pdf* (secciones “Identidad: ser uno mismo”, “Igual-

dad...”, “La igualdad y la identidad en Smalltalk”, “Protocolo e implementación”, “¿Qué es un método?”, “encapsulamiento”, y el “Resumen”: comportamiento + implementación + identidad). El polimorfismo se deduce de la combinación de la definición por protocolo + privacidad de la implementación allí descritas.

2. Repaso de subclasificación

2.1. La subclasificación como herramienta técnica y como herramienta de modelado conceptual

La herencia tiene **dos caras** que no hay que confundir:

- **Cara técnica (“mecánica”)**: la herencia me permite **reusar comportamiento e interfaz dinámicamente**, gracias al mecanismo de *method lookup*. Es la respuesta a la tentación de “copiar y pegar” código entre clases parecidas (p. ej. Guerrero y Espadachín): en lugar de duplicar, subclasifico.
- **Cara conceptual (modelado)**: subclasificar **representa una teoría del conocimiento** sobre una parte del dominio. **Sólo se debe subclasificar si es válido en el dominio**. La herramienta para decidirlo es la heurística “*todo X es un Y*” (en tanto y en cuanto X se comporte como Y, es decir, respete su protocolo).

El error clásico es usar la herencia *sólo* como mecanismo de reuso, ignorando la cara conceptual. El ejemplo de los guerreros muestra que reusar “porque ahorra código” puede llevar a jerarquías que no representan el dominio (p. ej. hacer Guerrero subclase de Espadachín, que reusa pero es conceptualmente falso: no todo guerrero es un espadachín).

Referencia: Clase 1 - Repaso herencia.pdf (slides “Herencia == Reusar código”, “¿Herencia == Reusar código?”, “Herencia: me permite reusar... / (!) Representa una teoría del conocimiento...”, “Heurística: todo X es un Y”, citando *The many faces of inheritance* de B. Meyer).

2.2. Algoritmo de búsqueda de método (*method lookup*)

Cuando un objeto recibe un mensaje, el sistema busca el método así:

1. Se parte de la **clase del objeto receptor**.
2. Si la clase define un método para ese selector, se ejecuta.
3. Si no, se sube a la **superclase**, y se repite la búsqueda.
4. El proceso continúa subiendo por la cadena de superclases hasta encontrar el método; si se llega a la cima (`BasicObject`) sin encontrarlo, se dispara el mecanismo de “no entiende” (en Ruby, `method_missing` / `NoMethodError`).

Este recorrido por la cadena de superclases es lo que permite el reuso dinámico de comportamiento (la “parte mecánica” de la herencia). En Ruby, además, los **módulos incluidos** (mixines) se insertan en esa cadena de búsqueda (ver Parte 2).

Referencia: Clase 1 - Repaso herencia.pdf (la herencia “permite reusar comportamiento e interfaz dinámicamente, dado el mecanismo del *method lookup*”); Clase 2 - Intro Metaprogramación.pdf y Clase 3 - Singleton classes...pdf (slides “Repaso: el proceso de búsqueda de métodos” / “*Method lookup*”, donde se muestra el recorrido instancia→clase→superclases hasta `BasicObject`).

2.3. Limitaciones de la herencia simple

La herencia simple (una clase, una única superclase) no alcanza para expresar clases que comparten *features* que no provienen de un ancestro común único. El ejemplo de los guerreros lo muestra: hay unidades que son **atacantes** (Fantasma, Guerrero), otras **defensoras** (Muralla, Guerrero) y combinaciones. Con herencia simple aparecen estos problemas:

- **Jerarquía forzada:** para reusar, se mete comportamiento “demasiado arriba” o se arman cadenas artificiales que no respetan el dominio (p. ej. `Defensor` subclase de `Atacante`), lo que obliga a anular/contradecir comportamiento heredado.
- **Código repetido:** si no se fuerza la jerarquía, hay que duplicar el código de “atacar” y de “defender” en las clases que lo necesitan.
- Otras alternativas exploradas en clase (anti-clases, delegación, *feature flags*) resuelven parcialmente pero tienen sus costos, lo que motiva mecanismos como **mixines** y la **herencia múltiple**.

La bibliografía generaliza esto: con herencia simple, las *features* compartidas o bien se fuerzan al ancestro común (donde no pertenecen, “polución” de la superclase) o bien se **duplican**. El ejemplo canónico es `ReadStream`, `WriteStream`, `ReadWriteStream`: `ReadWriteStream` necesita *features* de las dos pero sólo puede heredar de una, así que termina duplicando métodos o subiéndolos a `PositionableStream` (que queda contaminada).

Referencia: *Clase 1 - Repaso herencia.pdf* (slides “Jerarquía forzada”, “Repetir código”, “Uff!”, “El problema del diamante”); Ducasse et al., *Traits: A Mechanism for Fine-grained Reuse*, §2.1 “Decomposition Problems” (*Duplicated Features, Inappropriate Hierarchies*: ejemplo de los streams).

2.4. Interpretación y uso de `super`

`super` es el mecanismo por el cual una subclase, al redefinir un método, **invoca la versión heredada** del mismo selector. La sutileza clave del proceso de herencia es justamente la interpretación de `self` (recursión/auto-referencia) y de `super`.

En **Smalltalk** (y Ruby), `super` da acceso a los métodos del padre y la **subclase tiene el control**: puede agregar métodos, o reemplazar completamente un método del padre. No hay forma de que una superclase obligue a sus subclases a invocar su versión del método. Formalmente, una subclase se modela como una combinación asimétrica de un *delta* Δ (los cambios) sobre el padre P : $C = \Delta(P) \oplus P$, donde en caso de duplicación gana el valor de la izquierda (el de la subclase) y `super` permite que el delta acceda al método original de P .

Esto contrasta con **Beta**, donde la relación se **invierte**: el que controla es el *prefijo* (la superclase), y la extensión sólo puede insertarse en el punto donde el método del padre invoca `inner`. Es decir, *super de Smalltalk es análogo a inner de Beta, pero con la dirección de control invertida* (en Smalltalk decide la subclase; en Beta, la superclase).

Referencia: Bracha & Cook, *Mixin-based Inheritance*, §2.1 “Smalltalk Inheritance” (interpretación de `super`, ecuación $C = \Delta(P) \oplus P$) y §2.2–2.3 (comparación `super` vs. `inner`, “Smalltalk subclass refers to its parent using super just as a Beta prefix refers to its subpatterns using inner”).

Parte 2 — Esquemas no tradicionales para definir el comportamiento

3. Mixines

3.1. Mixin como función (de superclase a clase) / “subclase abstracta”

Un **mixin es una subclase abstracta**: una definición de subclase que puede aplicarse a **distintas superclases** para producir una familia de clases relacionadas. Matemáticamente, *un mixin es una función que toma una clase S (la superclase) y devuelve una subclase de S con un cuerpo de clase determinado*. Lo que distingue a un mixin de una subclase ordinaria es que **abstrae sobre la identidad de su superclase**.

El ejemplo clásico es `Comparable`, que abstrae sobre una superclase formal G y, asumiendo que G implementa `<`, define `≤`, `>`, `≥`, etc. Se invoca sobre distintas superclases concretas con el operador de

aplicación ▷:

```
ComparablePoint = Comparable ▷ Point ; Number = Comparable ▷ BasicNumber
```

Igual que con las funciones, el código fuente del mixin se **comparte** entre todas sus invocaciones (modularidad). Por eso *un mixin no es una clase*: hay que invocarlo sobre una superclase concreta para obtener una clase, así como una función hay que aplicarla a un argumento.

Referencia: Bak et al., *Mixins in Strongtalk*, §2 “Basic Model” (“a mixin is a function that maps a class S to a subclass of S”; ejemplo `Comparable` ; operador ▷). También Bracha & Cook, *Mixin-based Inheritance*, §3.2 (“A mixin is an abstract subclass”).

3.2. Requisitos que establece el mixin sobre su superclase

Como un mixin es una función (parcial), **no es aplicable a cualquier superclase**: impone **requisitos** sobre el parámetro, igual que una función no está definida para cualquier entrada. Por ejemplo:

- Si el mixin hace `super` -sends de ciertos métodos, esos métodos **deben estar definidos** en cualquier superclase concreta sobre la que se lo aplique.
- Si el mixin invoca métodos vía `self` que él no implementa (métodos *requeridos*), la superclase (o la clase resultante) debe proveerlos. `Comparable` requiere que la superclase implemente `<` (o `<=>` en Ruby).

Algunos de estos requisitos pueden expresarse con anotaciones de tipo. En Ruby, este “contrato” no es explícito: `Comparable` simplemente asume que la clase define `<=>`, y `Enumerable` asume que define `each`.

Referencia: Bak et al., *Mixins in Strongtalk*, §2 (“a mixin may place various requirements on the superclass... For example, a mixin may contain calls to super methods. These must be defined for any actual superclass”); *Programming Ruby* (cap. Modules/Mixins: “`Comparable` ... assumes that any class that uses it defines the operator `<=>`”; `Enumerable` requiere `each`).

3.3. Concepto de linearización. Interpretación de `super`

Cuando una clase combina varios mixins (o usa herencia múltiple estilo CLOS), el grafo de ancestros se **lineariza**: se transforma en una **cadena lineal** en la que cada ancestro aparece una sola vez, definiendo un orden total de búsqueda. La composición de mixins es justamente **lineal**: los mixins se aplican de a uno, generando subclases sucesivas en una jerarquía de herencia simple.

La interpretación de `super` en un mixin depende de esa linearización: como el mixin es una subclase abstracta cuyo padre se liga al aplicarlo, `super` **se liga tardíamente** (*late binding*) a la clase que quede inmediatamente “arriba” en la cadena linearizada. Esto es lo que da a los mixins su potencia: un mismo mixin puede, en distintas invocaciones, hacer `super` hacia distintas superclases. La linearización es, vista así, *la implementación* del mecanismo de aplicación de mixins: ubica al mixin en el lugar correcto de la cadena y puede **insertar otras clases entre el mixin y su padre**.

En Ruby esto se ve directamente: `include` no copia los métodos del módulo en la clase, sino que **inserta una referencia al módulo en la cadena de búsqueda de métodos**, justo por encima de la clase. Por eso los métodos mezclados “se comportan efectivamente como superclases”, y `super` dentro de la clase puede alcanzar a un método del módulo incluido.

Referencia: Bracha & Cook, *Mixin-based Inheritance*, §3.1–3.2 (linearización en CLOS; “linearization plays the role of application, by binding the mixin’s formal parent parameter to a specific class”; late-binding de `super` / `call-next-method`); *Programming Ruby* (Modules/Mixins: “mixed-in modules effectively behave as superclasses”; `include` hace una *referencia*, no copia).

3.4. Comparación con herencia múltiple

Los mixins se entienden mejor comparándolos con la herencia múltiple:

- **Herencia múltiple (p. ej. CLOS, C++):** varias clases padre se combinan directamente. Problema central: el “**problema del diamante**” (*Deadly Diamond of Death*) — cuando una clase hereda de un mismo ancestro por varios caminos, surgen ambigüedades (¿qué método gana?, ¿el estado se hereda una o varias veces?). CLOS lo resuelve linearizando, pero eso puede **violar el encapsulamiento** (cambia las relaciones padre-hijo declaradas) y un pequeño cambio en la jerarquía puede producir una linearización muy distinta.
- **Mixines:** usan el operador de herencia **simple** para extender distintas superclases con el mismo conjunto de features. Como se aplican de a uno (composición lineal), **no aparece el problema del diamante** ni conflictos ambiguos: cada mixin simplemente extiende o pisa lo de la clase a la que se aplica. El costo es que **el orden importa**: hay que encontrar un orden total adecuado de los mixins, que puede no existir, y suele aparecer “*glue code*” disperso por la jerarquía.

En resumen: los mixins ganan en simplicidad y evitan ambigüedades del diamante, pero pagan con la **dependencia del orden** de composición. (Esta tensión es la que motiva a los *traits*.)

Referencia: *Clase 1 - Repaso herencia.pdf* (slides “Solución con herencia múltiple”, “El problema del diamante”, “Solución con mixins”); Bracha & Cook, *Mixin-based Inheritance*, §3 (diamante, linearización, encapsulamiento); Ducasse et al., *Traits*, §2.2 “Composition Problems” (los mixins evitan conflictos pero su orden total puede no existir y dispersan *glue code*).

4. Traits

4.1. Definición de trait

Un **trait** es, en esencia, **un conjunto de métodos**, separado de toda jerarquía de clases (*pure units of reuse consisting only of methods*). Un trait **provee** métodos y puede **requerir** métodos (los que usa pero no implementa). La idea central es separar dos roles que en los lenguajes de clases están mezclados:

- las **clases** siguen siendo los **generadores de instancias** y se organizan en una jerarquía de **herencia simple**;
- los **traits** son **puras unidades de reuso**, ortogonales a la jerarquía.

Crucialmente, **los traits no especifican estado** (no tienen variables de instancia, sólo métodos). Por eso el único conflicto posible al componer traits es un **conflicto de métodos** (no de estado), que es mucho más fácil de resolver. Para acceder a estado, los traits originales lo hacen a través de *accessors* que quedan como métodos **requeridos**.

Referencia: Ducasse et al., *Traits*, §1 “Introduction” y §4 “Traits — Composable Units of Behavior” (“a trait is a set of methods, divorced from any class hierarchy”; “traits are purely units of reuse, and classes are generators of instances”; “Traits specify no state”).

4.2. Concepto de “flattening”, comparación con mixins, interpretación de **super**

La **propiedad de flattening** dice: *un método no sobre-escrito que viene de un trait tiene exactamente la misma semántica que si estuviera escrito directamente en la clase*. Es decir, los traits pueden “inlinearse” (aplanarse) en la clase sin cambiar el significado del programa: el hecho de que un método provenga de un trait y no de la clase **no afecta la semántica** de la clase.

Consecuencias e implicancias para **super**: como un método de trait se comporta como si estuviera definido en la clase, **super dentro de un trait se interpreta en el contexto de la clase que usa el trait**, no del trait. O sea, **super** apunta a la **superclase de la clase** (en la jerarquía de herencia simple), no a “otro trait”. Esto es coherente con que los traits estén divorciados de la jerarquía: el trait no introduce un nivel nuevo en la cadena de *method lookup*.

Comparación con mixins:

	Mixines	Traits
Unidad	subclase abstracta (parametrizada por superclase)	conjunto de métodos, sin estado
Composición	lineal / ordenada (de a uno)	no ordenada (varios a la vez)
Posición en la jerarquía	se insertan en la cadena de <i>lookup</i>	divorciados de la jerarquía (flattening)
Conflictos	resueltos por el orden (puede no haber orden válido)	el cliente los resuelve explícitamente
Estado	pueden arrastrar estado (colisiones de variables)	sin estado → sólo conflictos de método

La diferencia decisiva: **la composición de traits es no ordenada y el cliente (la clase) tiene el control total** sobre cómo se componen y cómo se resuelven los conflictos, evitando los problemas de linearización de los mixins.

Referencia: Ducasse et al., *Traits*, §4 (las tres reglas de composición: precedencia de la clase, **flattening property**, orden irrelevante) y §4.1 (“traits can be inlined, or flattened”; modelado de `selfSends` / `superSends`).

4.3. Separación de roles (clases vs. traits) según Ducasse et al.

El argumento de fondo de Ducasse et al. es que una **clase juega dos roles en conflicto**:

1. rol **primario**: ser **generadora de instancias** (por eso debe ser un paquete completo de features, sub-clase de `Object`);
2. rol **secundario**: ser **unidad de reuso** (por eso debería empaquetar un conjunto mínimo y coherente de features reusables juntas).

Estos roles entran en conflicto porque, al tener que ocupar una posición fija en la jerarquía, es difícil (i) factorizar *wrapper methods* como entidades reusables, (ii) resolver features en conflicto heredadas por distintos caminos, y (iii) acceder/componer features sobre-escritas. Los traits **resuelven el conflicto separando los dos roles**: la clase queda sólo como generadora de instancias, y la unidad de reuso pasa a ser el trait. Así, la herencia puede usarse para reflejar **jerarquía conceptual** (modelado) y no para reuso de código.

Referencia: Ducasse et al., *Traits*, §1 (“a class is primarily a generator of instances... A class has a secondary role as a unit of reuse... these two roles conflict”); §4 (introducción).

4.4. Reglas de composición, operadores y resolución de conflictos. La “ecuación” de las clases

Tres reglas de composición que respetan los traits:

1. **Precedencia de la clase**: los métodos definidos en la propia clase tienen prioridad sobre los provistos por un trait (las *glue methods* de la clase pueden pisar métodos del trait).
2. **Flattening**: un método de trait no sobre-escrito se comporta como si estuviera en la clase.
3. **Orden irrelevante**: todos los traits tienen la misma precedencia; por eso los métodos en conflicto **deben** desambiguarse explícitamente.

Operadores del modelo formal de traits (expresiones sobre traits):

- **Suma (+)**: combina traits tomando la unión de los métodos no conflictivos. Es **conmutativa**; cuando dos traits proveen el mismo nombre con cuerpos distintos (y no provienen del mismo trait), se produce un **conflicto**, representado mapeando ese nombre a $\top$.
- **Sobre-escritura / overriding (▷)**: mecanismo clave para resolver conflictos; un método de la clase (o un *glue method*) pisa el del trait.

- **Exclusión (-)**: quita un método de un trait (p. ej. para dejar el método en conflicto en uno solo de los traits).
- **Aliasing (→)**: da un **nombre adicional** a un método del trait, para poder acceder a un método que de otro modo quedaría inaccesible (p. ej. por haber sido sobre-escrito). Es menos frágil que el *renaming*.

Resolución de conflictos: un conflicto surge al combinar traits que proveen métodos con el mismo nombre que no vienen del mismo trait. Se resuelve (a) implementando un *glue method* en la clase que sobre-escriba a los conflictivos, o (b) **excluyendo** el método de todos los traits menos uno. El *aliasing* permite, además, recuperar acceso a un método tapado.

La “ecuación” de las clases (§4.3) resume cómo se arma una clase con traits:

$$\text{Class} = \text{Superclass} + \text{State} + \text{Traits} + \text{Glue methods}$$

Es decir: una clase se especifica componiendo una **superclase** (herencia simple), su **estado** (variables de instancia, que viven en la clase, no en los traits), un conjunto de **traits**, y las **glue methods** que conectan los traits entre sí (implementan los métodos requeridos, posiblemente accediendo al estado, adaptan métodos provistos y resuelven conflictos vía overriding/aliasing/exclusión).

Referencia: Ducasse et al., *Traits*, §4 (las tres reglas; operadores +, -, →, overriding); §4.2 (definición de conflicto, ⊥); §4.3 “Composing Classes from Traits” (la ecuación $\text{Class} = \text{Superclass} + \text{State} + \text{Traits} + \text{Glue methods}$). *Aclaración de la guía*: en el parcial no entra el formalismo estricto; alcanza con manejar estas ideas conceptualmente.

4.5. TP 1 y evolución posterior de los traits (¡los traits sí pueden tener estado!)

La guía remite al **enunciado y resolución del TP 1** como repaso práctico de traits (ya tienen el contexto de lo implementado). Además, advierte algo importante sobre la **evolución** del concepto: el equipo original implementó después **traits con estado interno** (*Stateful Traits*), mostrando que la ausencia de estado **no era una restricción esencial** del modelo, sino más bien una consecuencia de los desafíos de implementación en Smalltalk.

La idea de *Stateful Traits*: las variables de instancia son **locales al scope del trait** salvo que el cliente las haga explícitamente accesibles; así se evitan los conflictos de nombres de variables y se mantiene el principio rector de los traits (el **cliente retiene el control de la composición**). Por eso, **afirmar en el parcial que “los traits no pueden / no deben tener estado” sería un error**: pueden, y de hecho hay implementaciones modulares de traits con estado vía metaclasses especializadas.

Referencia: *Guía de estudio* (sección Traits, párrafo sobre el TP 1 y la advertencia sobre el estado); Bergel et al., *Stateful Traits* (Abstract e Introducción: traits originales son *stateless* “por la idiomática de accessors”, y se propone una extensión con estado donde el cliente controla la composición); Tesone et al., *A new modular implementation for Stateful Traits* (implementación vía metaclasses).

5. Closures

La guía aclara que **no hay bibliografía específica** sobre este tema en la carpeta: se vio a partir de la 3.^a clase (bloques) y se profundizó al programar el *framework de testing*. Por eso, además del contexto de la cátedra, se complementa con la **documentación oficial de Ruby** (única fuente externa permitida) para precisar el comportamiento de `proc` / `lambda`.

5.1. Concepto de closure (lambda abierta + contexto que la cierra)

Una **closure** es una **expresión lambda “abierta” junto con el contexto que la “cierra”**. Una lambda (función anónima) es “abierta” cuando hace referencia a variables que no son sus parámetros (variables libres); la closure es esa lambda **más** el entorno léxico que da significado a esas variables libres, “cerrándola”. En Ruby, los **bloques**, **Proc** y **lambda** son closures: capturan el entorno donde fueron creados.

Referencia: material de cátedra (clases sobre bloques y *framework de testing*); concepto estándar, consistente con la documentación oficial de Ruby sobre **Proc**.

5.2. Captura del contexto léxico y de **self**

Una closure captura su **contexto léxico**: las variables locales visibles en el lugar donde se definió siguen siendo accesibles cuando la closure se ejecuta después, incluso en otro contexto. Además, captura el valor de **self** vigente en su definición, de modo que dentro del bloque los mensajes “a sí mismo” se dirigen a ese **self** capturado (salvo que se cambie deliberadamente el contexto de evaluación, p. ej. con **instance_exec** — ver §8.2). Esta captura de **self** y del entorno es lo que permite, por ejemplo, que en el *framework de testing* un bloque de aserción ejecute en el contexto correcto.

Referencia: material de cátedra (uso de bloques y **self** en el *framework de testing*); documentación oficial de Ruby (**Proc** captura el binding léxico, incluido **self**).

5.3. Ligadura del **return** : **proc** (no local) vs. **lambda** (local). **LocalJumpError**

La diferencia central entre las dos clases de closures de Ruby está en **cómo se liga el **return**** :

- **proc / full closure (incluye los bloques)**: el **return** se liga de manera **no local**: un **return** dentro del bloque intenta retornar del **método que lo definió** (el método “dueño” del contexto), no sólo del bloque. Esto es un **retorno no local**.
- **lambda** : el **return** se liga de manera **local**: retorna sólo de la propia lambda, comportándose como una función común.

LocalJumpError : se produce cuando se intenta hacer un retorno no local desde un **proc** /bloque pero **el método que lo creó ya no está en la pila** (ya retornó). Como no hay a dónde “volver”, Ruby levanta **LocalJumpError**. (Típico: guardar un **proc** con **return** y llamarlo más tarde, fuera del método donde se creó.)

5.4. Para qué sirve el retorno no local: estructuras de control e “excepciones” no primitivas

Que se pueda ligar el **return** de forma no local es lo que **habilita implementar estructuras de control de manera no primitiva** (no *built-in* del lenguaje). El ejemplo a pensar es el **if** : si tenemos un **return** dentro del bloque de un **if**, esperamos que retorne del **método** que contiene al **if**, no que “retorne del if”. Esto sólo es posible si el bloque propaga el **return** de manera no local. Lo mismo permite implementar **excepciones** sin que sean primitivas del lenguaje.

En el **framework de testing** de la cátedra se usó precisamente un **retorno no local** para que, al fallar una aserción, las pruebas **dejaran de ejecutarse** a partir de ese punto **sin usar excepciones**.

Referencia: *Guía de estudio* (sección Closures: ligadura de **return**, **LocalJumpError**, *framework de testing*, implementación no primitiva del **if** y de excepciones); documentación oficial de Ruby (**Proc** vs. **lambda** : semántica de **return** y **LocalJumpError**) como única fuente externa, según lo habilita la propia guía.

Parte 3 — Metaprogramación

6. Introducción a la metaprogramación

6.1. Concepto de metaprograma. Ejemplos cotidianos

Un **metaprograma** es un **programa que genera, manipula o define a otros programas**. El prefijo *meta-* indica “acerca de”: un metaprograma es un programa “acerca de programas”. Ejemplos cotidianos de metaprogramas: **compiladores, formateadores de código, analizadores de código** (linters), **frameworks de testing, debuggers, refactorings automatizados y ORMs**.

Advertencia metodológica de la cátedra: **no hay que usar metaprogramación para resolver problemas que no pertenecen al dominio de la programación**. La metaprogramación es para problemas cuyo dominio es el software mismo.

Referencia: *Clase 2 - Intro Metaprogramación.pdf* (slides “Metaprogramas: programas que generan, manipulan o definen a otros programas”, lista de ejemplos; “(!) Recordar: no usar metaprogramación para problemas que no son del dominio de la programación”).

6.2. Reflexión: introspección (lectura) vs. intercesión (escritura)

La **reflexión** es la capacidad de un programa de **observar y, opcionalmente, modificar su propia estructura de alto nivel y su comportamiento**. Es un caso particular de metaprogramación: aquel en el que **metaprogramamos en el mismo lenguaje** en que están escritos los programas. Tiene dos modos:

- **Introspección:** habilidad del sistema de **razonar acerca de sí mismo** (es la parte de **lectura**). Ej.: preguntar a un objeto cuál es su clase, qué métodos entiende, etc.
- **Intercesión:** habilidad del sistema de **modificarse a sí mismo** (es la parte de **escritura**). Ej.: agregar/redefinir métodos, cambiar la clase, etc.

Referencia: *Clase 2 - Intro Metaprogramación.pdf* (slides “Reflexión”, “Introspección / Intercesión”).

6.3. Modelo vs. metamodelo

- **Modelo:** los objetos del dominio del problema y sus relaciones (p. ej. `atila` instancia de `Guerrero`, que es subclase de `Espadachín / Unidad`).
- **Metamodelo:** las **clases que representan a los conceptos del propio lenguaje** y sus relaciones — las que permiten *hablar del modelo*. En el metamodelo de objetos viven entidades como `Objeto`, `Clase`, `Método`, `superclase`, etc. El metamodelo es “el modelo del modelo”: describe de qué están hechos los objetos, clases y métodos.

Dicho de otro modo, cuando programamos resolvemos el problema en el **modelo**; cuando metaprogramamos, operamos sobre el **metamodelo**.

Referencia: *Clase 2 - Intro Metaprogramación.pdf* y *Clase 3.pdf* (slides “Modelos y metamodelos”: el metamodelo con `Objeto`, `Clase`, `Método`; el modelo con `Guerrero / Espadachín / atila`).

6.4. Sistemas reflexivos

Un **sistema reflexivo** es, según Maes, *un sistema que incorpora estructuras que representan (aspectos de) sí mismo*. A la suma de esas estructuras se la llama la **auto-representación** (*self-representation*) del sistema. Para que sea genuinamente reflexivo, la auto-representación debe estar **causalmente conectada** (*causally connected*) con el sistema: si una cambia, la otra cambia en consecuencia. De ahí dos propiedades:

1. el sistema **siempre tiene una representación precisa de sí mismo**, y
2. el estado y la computación del sistema están **siempre en conformidad** con esa representación (por eso el sistema puede modificarse a sí mismo por virtud de su propia computación).

Un lenguaje tiene **arquitectura reflexiva** si reconoce la reflexión como concepto fundamental: el intérprete da a cualquier programa acceso a datos que representan (aspectos de) el sistema, y **garantiza la conexión causal** entre esos datos y el sistema. En una arquitectura reflexiva, un sistema computacional se ve como compuesto por una **parte objeto** (resuelve el problema del dominio externo) y una **parte reflexiva** (resuelve problemas y devuelve información *acerca de la computación-objeto*).

Referencia: Maes, *Concepts and Experiments in Computational Reflection*, §2–4 (definiciones de *causally connected*, *reflective system*, *self-representation*, *reflective architecture*, *object part* / *reflective part*).

6.5. Reflexión procedural vs. reflexión declarativa

(Maes, §5.) La distinción tiene que ver con **cómo se da la auto-representación**:

- **Reflexión procedural:** la auto-representación se da en términos del **programa que implementa al sistema** (un intérprete meta-circular). Como esa representación *es* la que efectivamente corre el sistema, la conexión causal queda **automáticamente garantizada**: hay una sola representación, usada a la vez para implementar y para razonar sobre el sistema. Ejemplos: 3-LISP, BROWN. Inconveniente: la auto-representación procedural puede ser opaca/difícil de razonar.
- **Reflexión declarativa:** la auto-representación se da mediante **representaciones declarativas** (explícitas) acerca del sistema. La representación **implícita/procedural** sirve para *implementar* el sistema, mientras que la **explícita/declarativa** sirve para *computar acerca* del sistema. Permite representaciones más interesantes y fáciles de manipular, pero hay que **mantener la consistencia** entre la representación declarativa y el sistema (la conexión causal no es automática). Maes aclara que, más que una dicotomía, es un **continuo**.

Referencia: Maes, *Concepts and Experiments...*, §5 “Existing Reflective Architectures” (procedural reflection con intérpretes meta-circulares; declarative reflection; “the distinction... should more be viewed as a continuum”).

6.6. Propiedades deseables de un sistema reflexivo OO. Meta-objeto y *mirror*

Maes (§7, a propósito de 3-KRS) enumera **propiedades deseables** de un sistema reflexivo orientado a objetos:

1. **Separación objeto / meta-objeto:** la relación-meta **no se colapsa** con la relación de instanciación (a diferencia de Smalltalk/LOOPS). Cada objeto tiene un **meta-objeto** que lo representa, y objeto y meta-objeto están **físicamente separados**. Esta separación permite un **protocolo de mensajes estándar** entre un objeto y su meta-objeto, y poder *adjuntar* comportamientos meta especiales. (*Esta es la propiedad en la que se basa la idea de **mirror**.*)
2. **Uniformidad:** toda entidad del sistema es un objeto (instancias, clases, slots, métodos, meta-objetos, mensajes...), por lo que **cualquier aspecto puede ser reflejado**. Como los meta-objetos también son objetos, se construyen de forma *lazy* (perezosa).
3. **Compleitud:** la auto-representación contiene **toda** la información disponible sobre los objetos.
4. **Consistencia:** la auto-representación es **causalmente conectada** (se usa efectivamente para implementar el sistema, garantizando consistencia).
5. La auto-representación **puede a su vez tener meta-objetos** (torre reflexiva / lazy).

El concepto de **meta-objeto** (un objeto que representa la auto-representación correspondiente a otro objeto) está directamente relacionado con la idea de **mirror**: un objeto separado, a nivel meta, a través del cual se introspecciona/interviene sobre el objeto base, respetando la separación nivel base / nivel meta.

Referencia: Maes, *Concepts and Experiments...*, §7 (propiedades de 3-KRS: separación, uniformidad, completitud, consistencia; “meta-object”). La conexión con *mirror* se detalla en Bracha & Ungar (ver 6.7).

6.7. *Mirrors* (Bracha & Ungar) y la separación nivel base / nivel meta

El artículo *Mirrors* postula **tres principios de diseño** para las facilidades meta-nivel de cualquier lenguaje OO reflexivo; un sistema que los respeta es, por definición, *mirror-based*:

1. **Encapsulamiento (*Encapsulation*)**: las facilidades meta-nivel deben **encapsular su implementación**, de modo que el código cliente no dependa de detalles del sistema reflexivo (permite, p. ej., fuentes de datos locales o remotas sin cambiar el cliente).
2. **Estratificación (*Stratification*)**: las facilidades meta-nivel deben estar **separadas** de la funcionalidad de nivel base (se las puede agregar o quitar; una app desplegada puede prescindir de ellas). Esta es la realización a nivel de diseño de la **primera propiedad deseable de Maes** (separación base/reflexivo).
3. **Correspondencia ontológica (*Ontological correspondence*)**: la ontología de las facilidades meta-nivel debe **corresponderse** con la del lenguaje que manipulan (y, en su versión temporal, distinguir compile-time / load-time / run-time).

Un **mirror** es justamente un objeto del nivel meta a través del cual se reflexiona sobre un objeto base **sin** mezclar ambos niveles. Esto sirve para tener clara, **a nivel de diseño**, la necesidad de una **buena separación entre el nivel base y el nivel meta** de un sistema reflexivo (idea basada en la primera propiedad deseable de Maes).

Referencia: Bracha & Ungar, *Mirrors: Design Principles for Meta-level Facilities...*, §1–2 (los tres principios: *Encapsulation*, *Stratification*, *Ontological correspondence*; relación con la separación base/meta de Maes). Lectura opcional según la guía.

7. Aspectos estructurales de los metamodelos

7.1. Metamodelo de Ruby: partes principales y relaciones conceptuales

El metamodelo de Ruby está formado por (al menos) estas clases y relaciones:

- **BasicObject** — la raíz de la jerarquía de herencia (la clase más general; su superclase es `nil`).
- **Object** — subclase de **BasicObject**; raíz “práctica” de los objetos.
- **Kernel** — un **módulo** (mixin) incluido en **Object**, que aporta gran parte del protocolo común.
- **Module** — representa a los módulos; superclase de **Class**.
- **Class** — subclase de **Module**; representa a las clases. Toda clase es **instancia de Class** (relación “instancia de”, distinta de la relación “superclase”).
- Las clases del **modelo** del usuario (p. ej. **Guerrero**, **Espadachín**), que son instancias de **Class** y están conectadas por la relación de superclase, y sus **mixines** (**Atacante**, **Defensor**).

Dos relaciones conceptuales distintas estructuran el metamodelo: **“instancia de”** (un objeto y la clase que lo crea) y **“superclase”** (entre dos clases). En el diagrama del metamodelo se ve que **atila** es **instancia de Guerrero**; **Guerrero** es **instancia de Class** y su cadena de superclases sube por **Object** → **BasicObject**.

Referencia: *Clase 2 - Intro Metaprogramación.pdf* y *Clase 3 - Singleton classes...pdf* (slides “Parte del metamodelo de Ruby”: **BasicObject**, **Object**, **Kernel**, **Module**, **Class**, las flechas “instancia de” y “superclase”, * apunta a la clase **Class**).

7.2. Generalidades de la API de metaprogramación de Ruby

Sobre la API conviene saber:

- **Qué se puede hacer:** preguntar la clase de un objeto (`class`), su superclase, los métodos que entiende (`methods`, `instance_methods`), si responde a un mensaje (`respond_to?`), obtener métodos reificados (`method`, `instance_method`), definir métodos dinámicamente

- (`define_method`), reabrir clases, ejecutar bloques en distintos contextos (`instance_exec`, `class_exec`), enviar mensajes calculados (`send`), etc.
- **Distinguir mensajes reflexivos de no reflexivos:** un mensaje es **reflexivo** cuando opera sobre la estructura/comportamiento del propio sistema (p. ej. `class`, `define_method`, `superclass`), y **no reflexivo** cuando resuelve algo del dominio (p. ej. `atacar_a:`).
 - **Distinguir introspección de intercesión:** los mensajes de **introspección** sólo **leen** la auto-representación (`class`, `methods`, `respond_to?`, `instance_variables`); los de **intercesión** la **modifican** (`define_method`, `remove_method`, reabrir una clase, `singleton_class` + definir, etc.).

Referencia: *Clase 2/3.pdf* (slide “Generalidades de la API... poder distinguir cuándo un mensaje es reflexivo y cuándo no, cuándo es de introspección y cuándo no”); documentación oficial de Ruby para los mensajes concretos (única fuente externa, como habilita la guía).

7.3. Cosas para mejorar en el metamodelo de Ruby

Algunos puntos perfectibles que se discutieron:

- A diferencia de Smalltalk, Ruby **no reifica la metaclass** como concepto de primera clase: usa **singleton classes** (que son más generales pero “anónimas”/ocultas), y las metaclasses aparecen como un caso particular. Esto puede hacer menos transparente el razonamiento sobre el nivel de clase.
- La mezcla de **niveles** (clases que son objetos, pero el acceso meta no está estratificado al estilo *mirror*) implica que las facilidades meta no están separadas del nivel base (rompe la **estratificación** de Bracha & Ungar).
- Conviven dos relaciones (instanciación y herencia) que en algunos diseños se confunden; mantener su separación clara es parte de lo “mejorable”.

Referencia: *Clase 3 - Singleton classes...pdf* (discusión sobre singleton classes vs. metaclasses reificadas; metamodelo de Ruby); contrastes con Bracha & Ungar, *Mirrors* (estratificación) y con el metamodelo de Smalltalk (*Smalltalk y otros metamodelos.pdf*).

7.4. Singleton classes en Ruby. Metaclasses (Smalltalk) como caso particular

Una **singleton class** es una clase **propia y exclusiva de un único objeto**, donde viven los métodos definidos *sólo para ese objeto* (sus “métodos singleton”). En el *method lookup*, cuando un objeto recibe un mensaje, **primero se busca en su singleton class** y, si no está, se sigue por la cadena normal (la respuesta a `class` y sus superclases).

Regularidades del metamodelo (resumen de la Clase 3):

- **Todo objeto tiene una singleton class** (incluidas las clases, y por ende también las singleton classes tienen singleton class — de ahí la “torre”).
- **La superclase de la singleton class de un objeto es:**
 - la **clase del objeto**, si el objeto **no** es una clase;
 - la **singleton class de la superclase** del objeto, si el objeto **sí** es una clase (excepto para `BasicObject`).
- Por eso *el camino de búsqueda a partir de la singleton class agrega un prefijo (estricto) al camino que arranca en la respuesta a `class`*.

Las **metaclasses** de Smalltalk son un **caso particular** de singleton classes: en Smalltalk, la clase de una clase es su metaclass (reificada), que es exactamente la “singleton class de esa clase” donde viven los métodos de clase. Ruby generaliza la idea: las singleton classes funcionan para *cualquier* objeto, no sólo para clases, y los “métodos de clase” son simplemente métodos singleton de un objeto que resulta ser una clase.

Referencia: *Clase 3 - Singleton classes...pdf* (slides “¿Cómo entienden las clases mensajes de clase?”, “El camino a partir de la singleton class agrega un prefijo...”, “Algunas regularidades”,

“En resumen”); comparación con Smalltalk en *Smalltalk y otros metamodelos.pdf* (“tiene el concepto de metaclass reificado”).

7.5. La necesidad de una jerarquía paralela entre clases y metaclasses. Comunicación inter-nivel y compatibilidad

¿Por qué hace falta que las singleton classes / metaclasses formen una **jerarquía paralela** a la de las clases? Porque hay **comunicación inter-nivel**: los métodos de instancia mandan mensajes a su clase, y los métodos de clase mandan mensajes a las instancias. Para que esa comunicación no falle al heredar, los niveles deben mantenerse “alineados”. Esto es el **problema de compatibilidad** entre metaniveles, que justifica la jerarquía paralela (en la Clase 3 se ve que “la superclase de la singleton class de una clase es la singleton class de su superclase”, lo que **replica la jerarquía de clases un nivel arriba**).

Bouraqadi-Saâdani et al. (*Safe Metaclass Programming*) distinguen dos tipos de compatibilidad:

- **Compatibilidad ascendente (*upward compatibility*)**: sea `B` subclase de `A`, `MetaB` / `MetaA` sus metaclasses. Está asegurada si **todo mensaje que no produce error para instancias de `A`, tampoco produce error para instancias de `B`**. Surge porque un método de instancia puede enviar un mensaje *a la clase del receptor* (comunicación instancia→clase, p. ej. `self class ...`): para que `B` no falle, `MetaB` debe entender (implementar o heredar) ese mensaje de clase. Esto **exige que `MetaB` sea subclase de `MetaA`** → jerarquía paralela.
- **Compatibilidad descendente (*downward compatibility*)**: sea `MetaB` subclase de `MetaA`. Está asegurada si **todo mensaje que no produce error para `A`, tampoco lo produce para `B`**. Surge porque un método de clase puede enviar un mensaje *a una instancia recién creada* (comunicación clase→instancia, p. ej. `self new bar`): para que no falle, las instancias de `B` deben entender ese mensaje.

Como estos mensajes inter-nivel están embebidos en métodos, se **heredan** junto con ellos; asegurar la compatibilidad significa garantizar que esos mensajes siempre se entiendan. El artículo muestra que esto justifica la jerarquía paralela y, además, plantea problemas de este tipo de soluciones (la *class property propagation*), proponiendo luego un *compatibility model*.

Referencia: *Clase 3 - Singleton classes...pdf* (jerarquías paralelas; “la superclase de la singleton class de una clase es la singleton class de su superclase”); Bouraqadi-Saâdani, Ledoux & Rivard, *Safe Metaclass Programming*, §1–2 (compatibilidad ascendente y descendente, comunicación inter-nivel, justificación de la jerarquía paralela). Secciones 1 y 2 son las más importantes (la 2 es obligatoria).

8. Formas de representar y modificar el comportamiento dinámicamente

La guía aclara: **no se evalúa la sintaxis** (qué parámetros llevan estos mensajes, en qué orden), sino lo **conceptual** (por qué hace falta distinguir métodos ligados de no ligados, qué pasaría si sólo existiera `instance_exec`, etc.). Donde hace falta precisión sobre Ruby, se usa la documentación oficial.

8.1. `Method` y `UnboundMethod`: diferencia conceptual. `bind` / `unbind` y restricciones

Ruby **reifica los métodos** como objetos, en dos variantes:

- **`Method`** representa un método **ligado a un objeto receptor** (`self`). Como ya tiene un receptor, **se puede ejecutar** directamente enviándole el mensaje `call`.
- **`UnboundMethod`** representa un método **definido por una clase o mixin para sus instancias**, pero **todavía no ligado** a ningún receptor. Para ejecutarlo, primero hay que **ligarlo** (`bind`) a un objeto.

Restricciones de la ligadura (la parte conceptual importante):

- Si el `UnboundMethod` proviene de una **clase**, sólo se le pueden ligar **instancias de esa clase o de sus subclases** (porque el método podría depender del estado/protocolo garantizado por esa clase).
- Si proviene de un **mixin (módulo)**, **no hay restricciones** sobre a qué objeto se lo liga (un módulo no garantiza una jerarquía concreta, sólo el conjunto de métodos).

Por qué la distinción es necesaria (lo que pregunta la guía): separar “el método como pieza de comportamiento definida en una clase/módulo” de “el método ya asociado a un receptor concreto” permite **manipular, transplantar y reutilizar** comportamiento de forma controlada (p. ej. tomar un método de una clase y ejecutarlo sobre otro objeto compatible), manteniendo a la vez la **seguridad**: las restricciones de `bind` impiden ejecutar un método sobre un objeto que no cumple las precondiciones de la clase que lo definió. `unbind` hace el camino inverso: obtiene el `UnboundMethod` a partir de un `Method`.

Referencia: *Clase 2 - Intro Metaprogramación.pdf* (slide “Method vs UnboundMethod”: `UnboundMethod` no está ligado, hay que ligarlo; si viene de una clase sólo se ligán instancias de esa clase o subclases; si viene de un mixin no hay restricciones; `Method` se ejecuta con `call`). Detalles de `bind` / `unbind`: documentación oficial de Ruby.

8.2. `instance_exec` vs. `module_exec` / `class_exec` : por qué existen ambas

Ambas pertenecen a la familia `eval` / `exec` y sirven para **evaluar un bloque en un contexto distinto** del de su definición, pero **difieren en dos cosas correlacionadas: quién es `self` y dónde se definen los métodos**:

- **`instance_exec`** : evalúa el bloque en el contexto de **un objeto**. Dentro del bloque, **`self` es el objeto receptor** del `instance_exec`. Los métodos que se definan dentro se definen en la **singleton class del objeto receptor** (es decir, métodos *de ese objeto particular*).
- **`class_exec` / `module_exec`** : evalúan el bloque en el contexto de **una clase o módulo**. Dentro del bloque, **`self` es la clase/módulo receptor**, y los métodos definidos se agregan a la **clase/módulo receptor** (es decir, pasan a ser **métodos de instancia** de esa clase / del módulo).

Por qué existen ambas (lo conceptual): se necesitan porque corresponden a **dos intenciones distintas** sobre **dónde** va a vivir el comportamiento que se define dinámicamente. Si **sólo** tuviéramos `instance_exec`, podríamos definir comportamiento para **un objeto puntual** (en su singleton class), pero **no** podríamos agregar de forma directa **métodos de instancia compartidos por todas las instancias** de una clase: tendríamos que recurrir a rodeos (operar sobre la clase tratándola como objeto y, aun así, terminar en su singleton class, que da métodos *de clase*, no *de instancia*). Es decir, sin `class_exec` perderíamos la capacidad de modificar el comportamiento **de todas las instancias** a la vez de manera limpia. Las dos variantes existen porque “el contexto de un objeto cualquiera” y “el contexto de una clase como definidora de métodos de instancia” son **niveles distintos** del metamodelo.

Referencia: *Clase 3 - Singleton classes...pdf* (slide “Familia de métodos eval/exec”: `instance_exec` → `self` es el objeto receptor, métodos en la singleton class del receptor; `class_exec` / `module_exec` → `self` es la clase/módulo receptor, métodos en la clase/módulo). La guía menciona expresamente como ejemplo conceptual “qué pasaría si solamente tuviéramos `instance_exec` pero no `class_exec`”.

8.3. Clases abiertas, `define_method` y cambio dinámico del comportamiento

- **Clases abiertas**: en Ruby las clases se pueden **reabrir** en cualquier momento (volver a escribir `class X ... end`, incluso en ejecución) para **agregar, redefinir o quitar** métodos. Como `include` y los métodos viven por **referencia**, redefinir un método de un módulo o clase en tiempo de ejecución hace que **todas** las instancias/clases que dependen de él **exhiban el nuevo comportamiento** inmediatamente. Esto vale incluso para las clases *built-in* (en *Programming Ruby* se reabre `Array` y `Range` para mezclarles el módulo `Inject`).

- **define_method**: permite **definir un método nuevo dinámicamente** pasándole un **bloque** (closure) como cuerpo. A diferencia de **def**, como recibe un bloque, el método **captura el contexto léxico** donde se creó, y su nombre puede **calcularse** en tiempo de ejecución. Es una herramienta de **intercesión**.
- **Cambio dinámico del comportamiento de las instancias**: combinando clases abiertas + **define_method** + singleton classes, se puede cambiar **en ejecución** qué entiende un objeto o una clase: agregar métodos a todas las instancias (reabriendo la clase / **class_exec**) o sólo a un objeto puntual (su singleton class / **instance_exec**).

Comentario conceptual (“¿clases como moldes?”): pensar la clase como un “molde” estático que estampa instancias es engañoso en Ruby, porque la clase es un objeto vivo y modificable: cambiar la clase repercute sobre instancias **ya creadas**, cosa que un molde no permite. La clase es más bien un **objeto que decide el comportamiento** de sus instancias mediante el *method lookup*, y ese comportamiento puede cambiar dinámicamente.

Referencia: *Clase 3 - Singleton classes...pdf* (slides “¿Qué herramientas nos provee Ruby para definir comportamiento?”: clases abiertas, **define_method** (necesita un bloque), “Comentario: ¿clases como moldes?”, bloques como reificación de colaboraciones); *Programming Ruby* (cap. Mixins: **include** hace referencia, no copia → cambios dinámicos; reapertura de **Array / Range**). Detalles de **define_method**: documentación oficial de Ruby.

9. Comparación de metamodelos y maneras de entender la programación

9.1. Ruby vs. Smalltalk vs. Java: ¿qué gana y qué pierde cada uno?

Smalltalk (extremo “todo es mensaje / sistema vivo”):

- (Casi) **todo se resuelve enviando mensajes**, incluso el control de flujo: el **if** es un mensaje (**ifTrue:ifFalse:**) a un booleano.
- Se programa en un **ambiente vivo**, modificando el sistema en **tiempo de ejecución**.
- Tiene el concepto de **metaclase reificado** (de primera clase).
- **Gana**: máxima uniformidad y coherencia con el paradigma, reflexión potente, expresividad. **Pierde**: puede ser menos familiar/eficiente y exige todo el ambiente.

Java (extremo opuesto, “muchas cosas fuera del paradigma”):

- Muchas construcciones **se van del paradigma**: los **números no son objetos** (primitivos), los **arrays no son colecciones**, los **operadores aritméticos** y **new** no son mensajes.
- **Metamodelo mínimo**: las clases se representan únicamente con la clase **Class**. En consecuencia, los “**métodos estáticos**” **no pueden ser métodos de verdad** → **no hay polimorfismo** sobre ellos, **no hay this** en ese contexto, etc.
- **Gana**: simplicidad de implementación, performance, tipado estático, familiaridad. **Pierde**: uniformidad, expresividad y poder reflexivo; rompe el paradigma en varios puntos.

Ruby está “**en el medio**” del espectro: casi todo es objeto y hay reflexión potente (como Smalltalk), pero conserva sintaxis y construcciones de lenguajes más tradicionales y no reifica la metaclase (usa singleton classes). Gana pragmatismo y familiaridad sin abandonar del todo la uniformidad; pierde algo de la coherencia total de Smalltalk.

Referencia: *Smalltalk y otros metamodelos.pdf* (slides “Estructura del metamodelo” de Smalltalk con **Metaclass / Metaclass class**; “Java” con metamodelo mínimo **Object / Class**); *Guía de estudio* (cuadro Smalltalk/Java/Ruby “en el medio”).

9.2. “Lenguajes” vs. “sistemas/ambientes” de programación

Dos maneras de entender la programación:

- **Centrada en el lenguaje:** el programa es un **artefacto lingüístico, estático** (texto fuente). El foco está en la gramática, las palabras reservadas, lo que se puede escribir.
- **Centrada en el sistema/ambiente:** el programa es un **proceso en ejecución, dinámico**, un sistema vivo que se inspecciona y modifica mientras corre (visión típica de Smalltalk).

Esta distinción explica por qué, si pensamos en términos estáticos, tendemos a “programar orientados a clases” (pensar en el texto, en las clases) en lugar de “orientados a objetos” (los objetos sólo existen en ejecución).

Referencia: *Smalltalk y otros metamodelos.pdf* (“Explorar la diferencia entre... centrada en el lenguaje, y centrada en el sistema”); *Guía de estudio* (sección “Comparación de metamodelos...”: programa como artefacto lingüístico estático vs. proceso en ejecución dinámico); nota al margen del Apéndice sobre “orientado a clases vs. orientado a objetos”.

9.3. Resolución “paradigmática” vs. “sintáctica”. Sintaxis flexible (Ruby)

- **Manera paradigmática:** resolver un problema usando **sólo los conceptos del paradigma** (objetos y mensajes). Ej.: crear una clase con `X = Class.new` — usando los elementos básicos del paradigma.
- **Manera sintáctica:** **agregar sintaxis específica** (palabras reservadas, construcciones especiales) para resolver lo mismo. Ej.: `class X; end` — azúcar sintáctica.

Ruby es interesante porque **habilita ambas opciones** y ofrece una **sintaxis flexible** que permite “expresarnos como queremos sin agregar palabras reservadas”. Esto conecta con la idea, vista en el TP 1, de la “**interfaz de usuario**” de un metaprograma (la sintaxis amena que se le ofrece al usuario) en contraste con el **modelo interno** (los objetos y mensajes que la implementan).

Lo importante, según la guía, **no es tomar partido**, sino haber desarrollado un **criterio** para distinguir estos enfoques y poder posicionarse de manera informada, **con argumentos y justificaciones**.

Referencia: *Guía de estudio* (sección “Comparación de metamodelos...”: resolución paradigmática vs. sintáctica; sintaxis flexible; `class X; end` vs. `X = Class.new`; “interfaz de usuario” del metaprograma vs. modelo interno; lo importante es el criterio y los argumentos); *Smalltalk y otros metamodelos.pdf*. Lectura relacionada (no obligatoria): Gabriel, *The Structure of a Programming Language Revolution*.

Apéndice — Comentarios sobre la subclasificación (resuelto)

La heurística para decidir si `X` debe ser subclase de `Y` es “**todo X es un Y**” (en tanto `X` se comporte como tal, es decir, respete el protocolo). La precisión clave: hay que decir “**todo _ es un**” y no simplemente “**es un**”, porque “*es un*” confunde **una relación entre clases** con **una relación entre una clase y sus instancias**.

Esto se apoya en distinguir, en el **dominio**, dos relaciones (que en el programa reificamos como clases —conceptos— y objetos —cosas—):

1. **Relación de generalización** (entre **conceptos**): se corresponde con la relación **subclase–superclase**. También se la llama *subsunción* o *subordinación conceptual*. Equivale a la **inclusión** entre conjuntos: $\text{ConceptoEspecífico} \subseteq \text{ConceptoGeneral}$.
2. **Relación entre una cosa y un concepto bajo el cual cae** (entre **un individuo y un concepto**): se corresponde con la relación **clase–instancia**. También se la llama *ejemplificación* o “*es un*” (pepita *es una* golondrina). Equivale a la **pertenencia**: $\text{individuo} \in \text{Concepto}$.

El vínculo entre ambas: un concepto es más general que otro **si y sólo si** todas las cosas que caen bajo el específico caen bajo el general:

$$\text{ConceptoEspecífico} \subseteq \text{ConceptoGeneral} \iff (\forall \text{individuo} \in \text{ConceptoEspecífico}) \text{individuo} \in \text{ConceptoGeneral}.$$

Conclusión: “**es un**” es en realidad la relación **instancia–clase** (cosa–concepto), no la relación entre dos clases. Si decimos “**todo _ es un**”, el “todo” **cuantifica sobre las instancias** sin referirse a una en particular, y entonces sí estamos hablando de una **relación entre clases** (subclasificación).

Comentario al margen (confusión clase/objeto): este desliz se relaciona con que a veces, en lenguajes basados en clases, terminamos programando “**orientados a clases**” en lugar de “orientados a objetos” — porque si nos importa el programa como artefacto **estático**, pensamos en clases (los objetos sólo existen en ejecución). Ejemplo: la “S” de SOLID, “una clase debería tener una única razón para cambiar”, aplicada literalmente con una cadena de subclases, *cumple la letra* pero deja al objeto con la misma baja cohesión de antes.

Referencia: *Guía de estudio*, Apéndice “Comentarios sobre la subclasificación” (heurística “todo X es un Y”; relación de generalización/subsunción $\subseteq$ vs. ejemplificación/“es un” $\in$; la equivalencia con la inclusión y la pertenencia; nota sobre “orientado a clases” y la “S” de SOLID del cap. 8 de R. Martin).

Lista de lecturas (referencia rápida)

Obligatorias

- Maes, P. (1987). *Concepts and Experiments in Computational Reflection*. OOPSLA ’87. (Secciones 1–4; §5 procedural vs. declarativa; §7 propiedades deseables.)
- Bouraqadi-Saâdani, Ledoux & Rivard (1998). *Safe Metaclass Programming*. OOPSLA ’98. (§2 obligatoria; compatibilidad ascendente/descendente.)
- Bracha, G. & Cook, W. (1990). *Mixin-based Inheritance*. OOPSLA/ECOOP ’90. (Todo salvo §5; §2 super vs. inner; §3 linearización.)
- Ducasse, Nierstrasz, Schärli, Wuyts & Black (2006). *Traits: A Mechanism for Fine-grained Reuse*. ACM TOPLAS 28(2). (§1, §2, §4 núcleo y §4.3 ecuación de clases.)

Opcionales

- Bracha, G. & Ungar, D. (2004). *Mirrors: Design Principles for Meta-level Facilities...* OOPSLA ’04. (Tres principios; separación base/meta.)
- Bak, Bracha, Grarup, Griesemer, Griswold & Hölzle (2002). *Mixins in Strongtalk*. (§2 resumen de mixin como función.)
- Bergel, Ducasse, Nierstrasz & Wuyts (2006). *Stateful Traits*. (Traits con estado.)
- Tesone, Ducasse, Polito, Fabresse & Bouraqadi (2020). *A new modular implementation for Stateful Traits*.

Documentación oficial de Ruby — usada sólo donde la cátedra no proveyó bibliografía específica (closures: `proc` vs. `lambda`, `LocalJumpError`; `bind/unbind`; `define_method`; `instance_exec/class_exec`), conforme lo habilita la guía.